

M. Nivedha
(192471040)

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: COMPUTER ARCHITECTURE **COURSE CODE:** CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

**ANNEXURE A
APPLICATION BASED QUESTIONS**

85% 

Code of Conduct:

I M. Nivedha (192471040) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

M. Nivedha
Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch-decode-execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes

of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)	
Q1	4	5	4	3	2	18
Q2	3	6	3	4	2.5	18.5
Q3	4	6	4	4	2	20
Q4	3	5	4	3	1	16
Q5	2	4	2	2	2.5	12.5

85

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements ⁴	Good understanding with minor gaps ³	Basic understanding; some requirements unclear ²	Poor understanding or misinterpretation of the problem ¹
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems ⁶	Applies relevant concepts correctly with minor errors ⁵	Applies basic concepts with limited accuracy ⁴	Fails to apply appropriate concepts ¹⁻²
Problem-Solving Approach	20	Logical, well-structured, and efficient approach ⁴	Mostly logical approach with minor gaps ³	Basic approach with some inconsistencies ²	Illogical or unclear approach ¹
Accuracy of Solution	20	Completely correct solution with proper justification ⁴	Mostly correct with minor errors ³	Partially correct solution ²	Incorrect or incomplete solution ¹
Clarity	10	Clear, well-organized, and properly explained solution ²	Understandable with minor clarity issues ^{1.5}	Limited clarity and explanation ¹	Poorly presented and difficult to understand ⁰

1. 1. Smart City Surveillance System – CPU, Memory, I/O and Bus Structure

Understanding the Problem

A smart city surveillance system continuously receives video data from multiple cameras. During peak hours, huge amounts of video data are transferred between I/O devices, memory, and the CPU. Since all these components communicate simultaneously, the processor becomes overloaded, causing delays in threat detection.

Application of Concepts

CPU

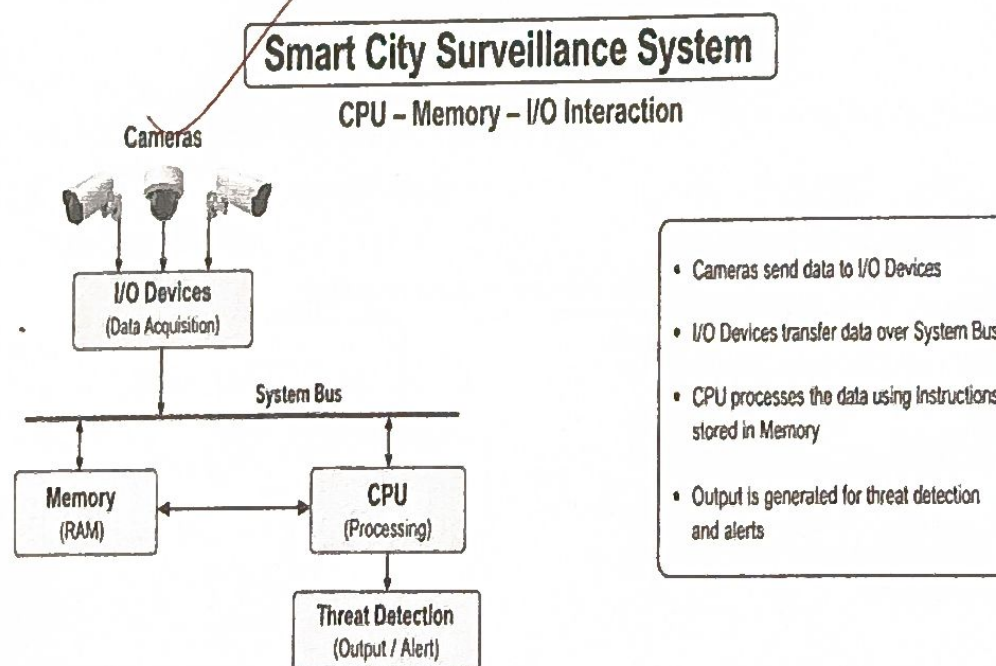
- Processes video frames using image processing algorithms.
- Identifies suspicious activities.
- Generates alerts for security personnel.

Memory

- Stores incoming video frames temporarily in RAM.
- Provides fast access to processing data.
- Slow memory access increases waiting time.

Input/Output Devices

- Cameras continuously send video data.
- Storage devices save recordings.
- Display units show live surveillance.



Analysis of Bus Structures

Single Bus Architecture

- CPU, memory, and I/O share one communication bus.
- Only one transfer occurs at a time.
- Heavy traffic creates bus contention.
- CPU waits for data transfer, increasing delay.

Multiple Bus Architecture

- Separate buses are provided for memory and I/O.
- Multiple transfers occur simultaneously.
- Reduces waiting time.
- Improves overall surveillance performance.

Improving Instruction Execution

- Use instruction pipelining.
- Increase cache memory.
- Apply Direct Memory Access (DMA) for camera transfers.
- Use multi-core processors for parallel processing.

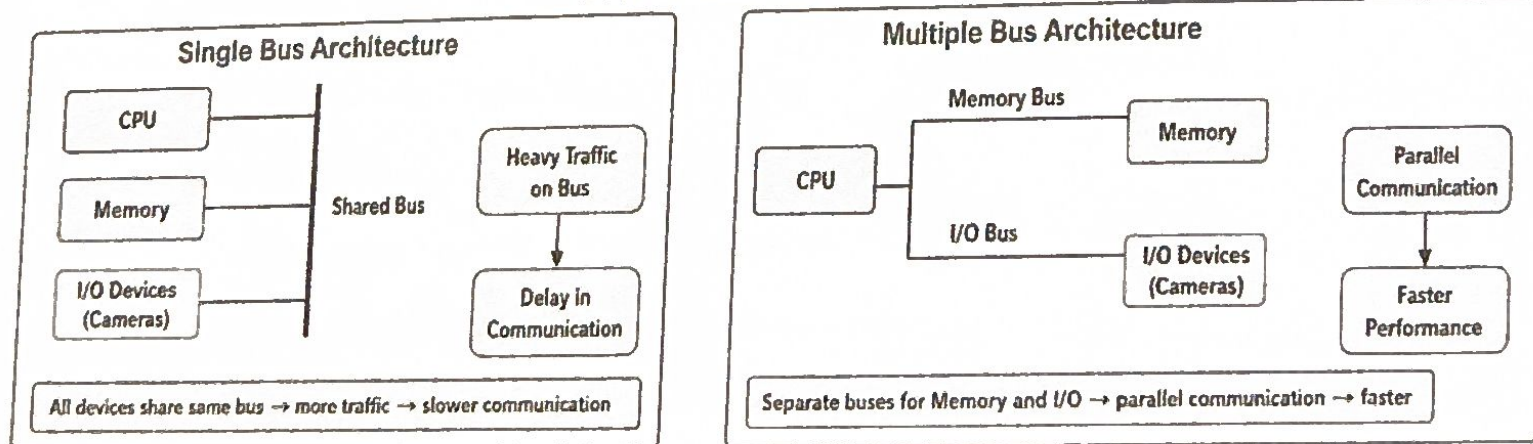
Solution

A multiple-bus architecture combined with pipelining, cache memory, and DMA minimizes communication delays and improves real-time threat detection.

Conclusion

Efficient coordination between CPU, memory, I/O devices, and multiple buses enables faster video processing and reliable smart city surveillance.

Bus Architecture Comparison



Real-Time Stock Trading Application – Fetch–Decode–Execute Cycle and CPI

Understanding the Problem

A stock trading application executes millions of instructions every second. Frequent memory accesses and branch instructions increase the Clock Cycles Per Instruction (CPI), causing transaction delays.

Application of Fetch–Decode–Execute Cycle

➤ Fetch

The processor fetches the next trading instruction from memory.

➤ Decode

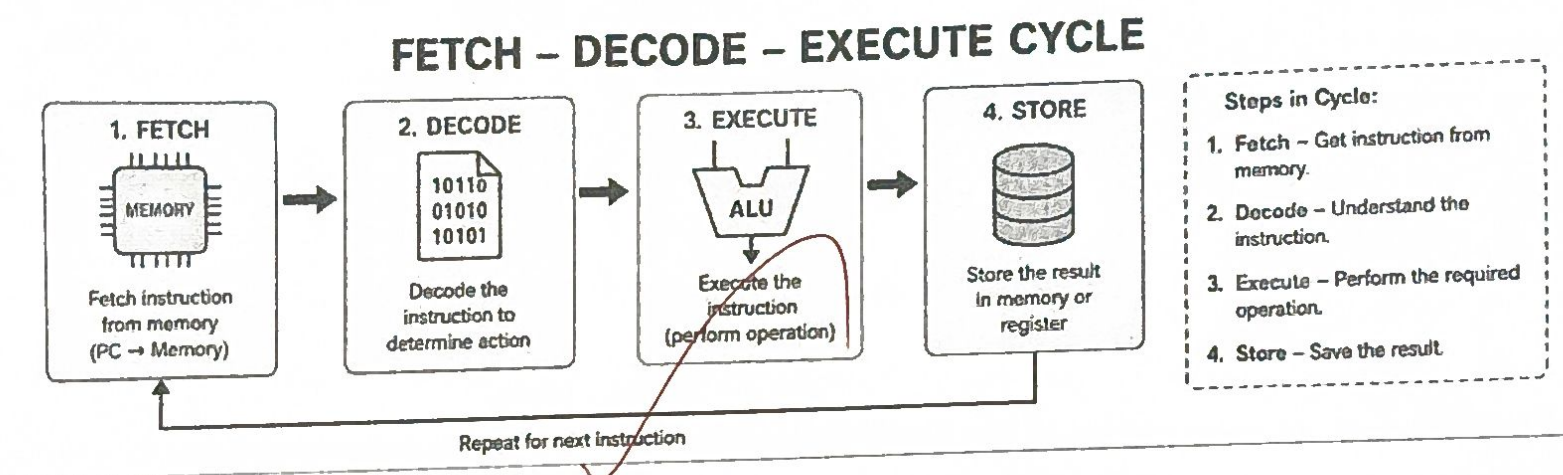
The control unit interprets the instruction.

➤ Execute

The ALU performs calculations such as price comparison and order validation.

➤ Store

The processed transaction is stored in memory.



Frequent memory accesses increase fetch time, while branch instructions interrupt smooth execution.

Impact on Execution Time

- High CPI increases execution time.
- Branch instructions cause pipeline stalls. Memory delays reduce CPU efficiency.
- Users experience slower transaction processing.

Methods to Reduce CPI

- Increase cache memory to reduce memory access.
- Use pipelining for simultaneous instruction execution.
- Apply branch prediction techniques.
- Use superscalar processors.
- Optimize software algorithms.

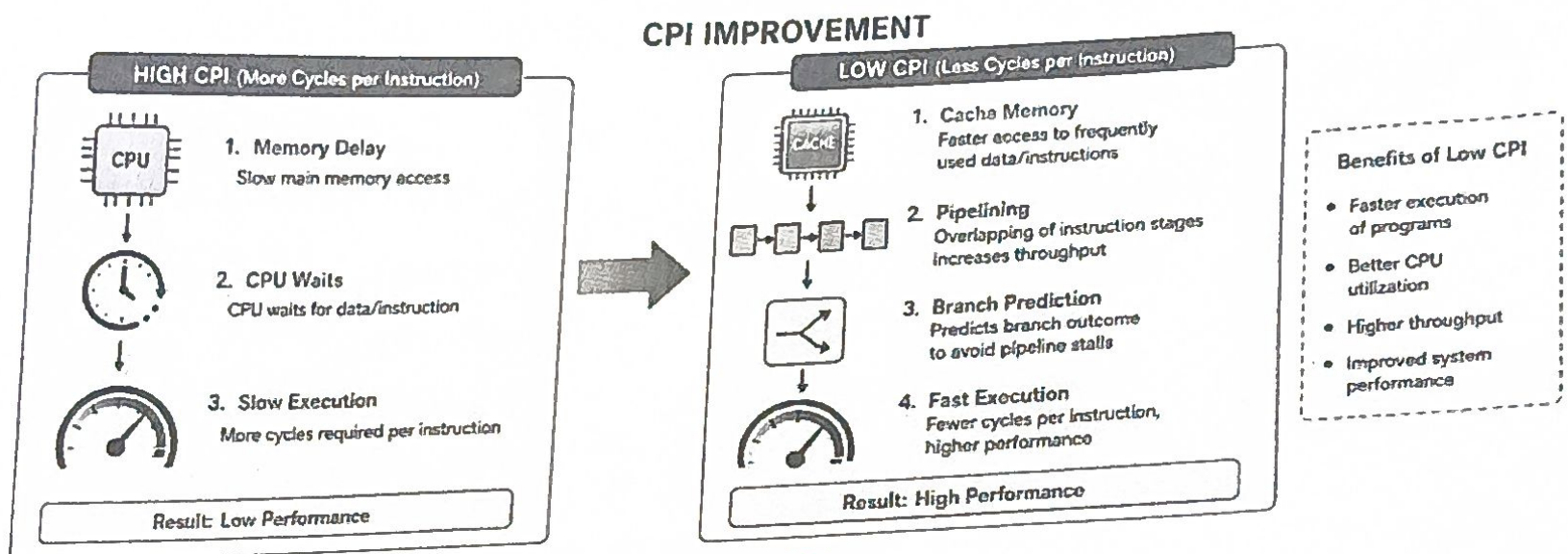
Employ multi-core processors.

Solution

Reducing CPI through cache optimization, branch prediction, pipelining, and parallel execution significantly improves transaction speed.

Conclusion

Lower CPI results in faster instruction execution, reduced processing delay, and improved performance of real-time trading systems.



Q3. 8085 Temperature Control System – Addressing Modes

Understanding the Problem

An industrial temperature controller receives sensor values and controls actuators. Incorrect addressing modes sometimes cause wrong temperature readings, leading to improper control decisions.

Application of Addressing Modes

Immediate Addressing

Stores constant values such as temperature limits.

➤ Example:

MVI A,40H

Register Addressing

Transfers data between CPU registers.

➤ Example:

MOV A,B

Direct Addressing

Reads temperature from a fixed memory location.

➤ Example:
LDA 2050H

Register Indirect Addressing

Uses HL register pair to access sensor data.

➤ Example:

MOV A,M

Implied Addressing

Performs operations without specifying operands.

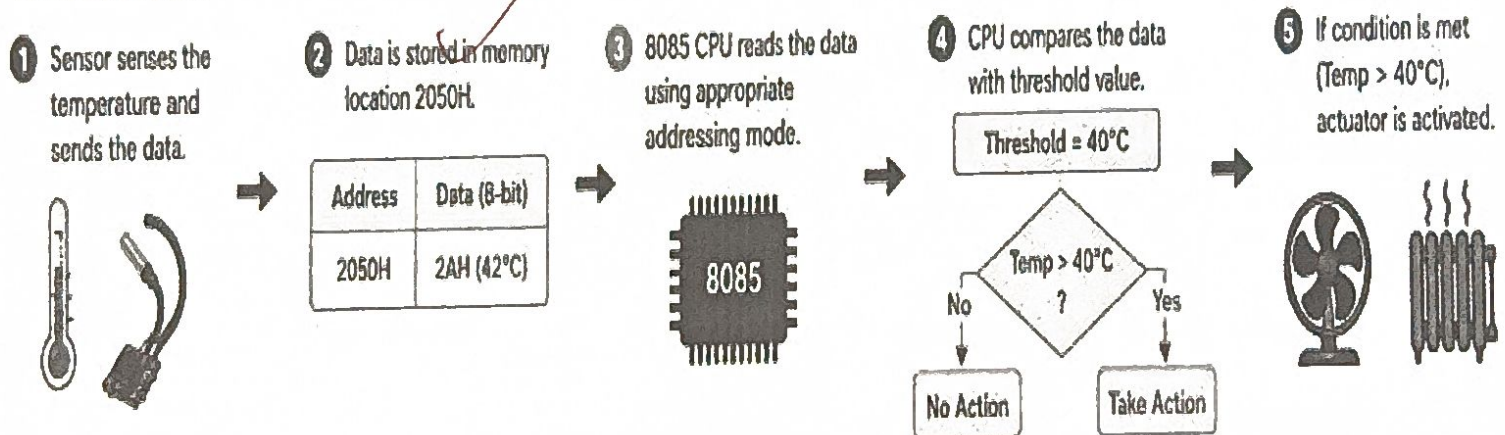
➤ Example:

CMA

Causes of Incorrect Readings

- Wrong memory address selected.
- Incorrect HL register value.
- Invalid sensor data location.
- Improper register usage.
- Programming mistakes in assembly code

WORKING STEPS



Using 8085 Instruction Set Effectively

- Use LDA to read sensor values correctly.
- Store processed data using STA.
- Compare readings using CMP.
- Use conditional jumps like JC and JZ.
- Validate sensor values before actuator control.

Solution

Correct addressing modes ensure the CPU accesses the intended sensor data, reducing errors and improving system reliability.

Conclusion

Proper selection of addressing modes and correct use of the 8085 instruction set ensure accurate temperature monitoring and safe industrial operation.

Q4. 8086 Memory Segmentation

Understanding the Problem

A hospital management system running on an 8086 processor stores patient records and billing data. Due to incorrect memory segmentation, the system accesses wrong data and causes stack overflow during function calls.

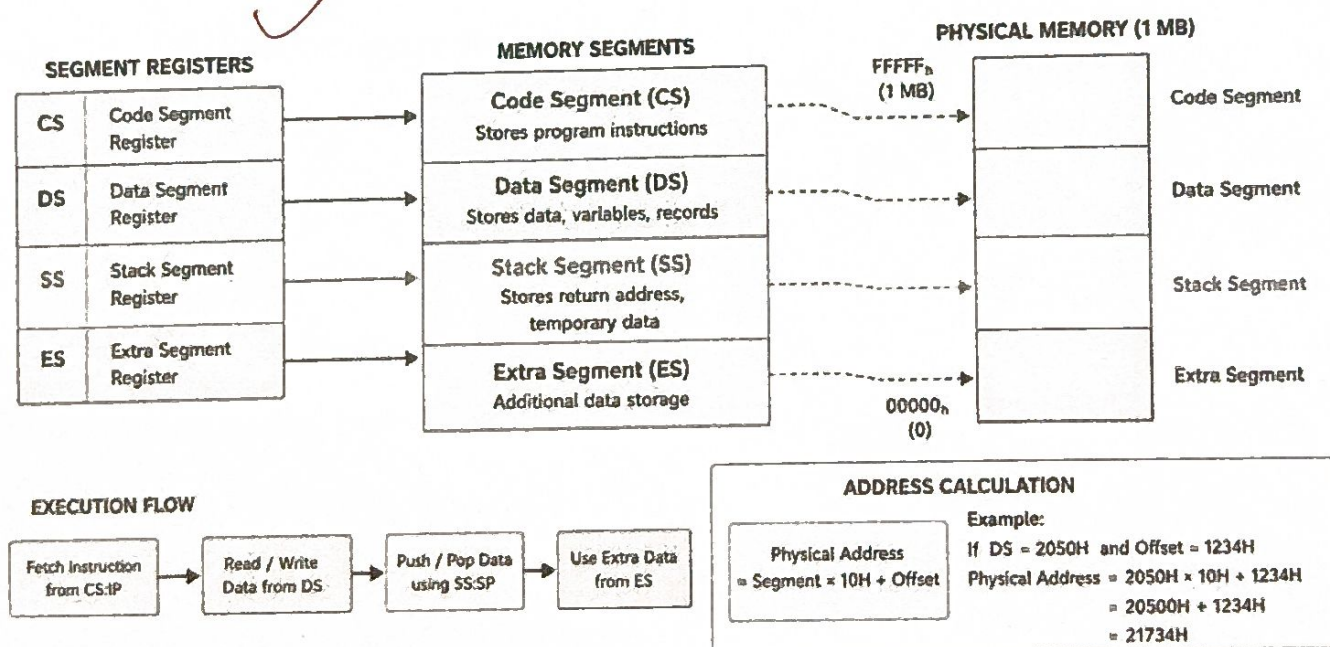
Application

The 8086 divides memory into four segments:

- **Code Segment (CS):** Stores program instructions.
- **Data Segment (DS):** Stores patient records and billing data.
- **Stack Segment (SS):** Stores return addresses and temporary data.
- **Extra Segment (ES):** Stores additional data when required.

Physical Address:

$$\text{Physical Address} = \text{Segment} \times 10\text{H} + \text{Offset}$$



Causes of Errors

- Wrong segment register values.

- Stack overflow due to excessive PUSH operations.
- Invalid memory offsets.
- Mixing code and data segments.

Solution

- Initialize CS, DS, SS, and ES correctly.
- Keep code, data, and stack in separate segments.
- Maintain the Stack Pointer (SP).
- Use balanced PUSH and POP instructions.

Conclusion

Proper memory segmentation ensures correct data access, prevents stack overflow, and improves system reliability and execution efficiency.

Q5. Number System Conversion and CPU Performance

Understanding the Problem

A communication system uses hexadecimal packet data. Incorrect hexadecimal-to-binary conversion causes the processor to interpret packet information incorrectly.

Importance of Hexadecimal and Binary

- Binary is understood directly by digital circuits.
- Hexadecimal represents binary in a compact form.
- One hexadecimal digit equals four binary bits.
- Easier debugging and memory representation.

Effects of Conversion Errors

- Incorrect instruction decoding.
- Wrong memory addresses.
- Invalid packet interpretation.
- Data corruption during transmission.
- Communication failures.

CPU Design to Minimize Errors

Arithmetic Logic Unit (ALU)

Processes binary calculations.

Control Unit

Registers temporarily store converted values.

Cache Memory

Provides faster data access.

Benchmarking

CPU benchmarking helps identify:

- Processing delays.
- Conversion errors.
- Memory bottlenecks.
- Execution efficiency.
- Overall system performance.

Solution

- Validate hexadecimal-to-binary conversion.
- Use automated conversion routines.
- Perform error checking before execution.
- Benchmark CPU performance regularly.
- Optimize cache and instruction processing.

Conclusion

Accurate number system conversion combined with efficient CPU design and benchmarking ensures reliable data communication and high system performance

